

OpenGL 기초

2026년 2학기

OpenGL 이란

• OpenGL이란

- 1982년 Silicon Graphics(SGI)사의 워크스테이션용 그래픽스 라이브러리 아이리스 지엘 (Iris GL) 로 시작하여 1992년 OpenGL 1.0으로 출시된 2D 및 3D 그래픽스 API
- GPU에게 “무엇을 그려라”라고 명령을 내리기 위한 그래픽스 API 명세 (specification)
- 300개 이상의 함수 호출을 이용하여 단순한 기하도형에서부터 복잡한 3차원 장면 생성
- 창을 띄우거나 키보드 입력을 받는 기능은 전혀 제공하지 않고 오직 “그리기”에만 관여
 - 윈도우/입력 처리: GLFW
 - 최신 OpenGL 함수 로딩: GLEW
 - 수학적 내용: GLM



- 그래픽스 API (Application Programming Interface): 프로그램이 GPU에게 그래픽 작업을 시키기 위한 약속된 명령체계
 - API: 프로그램이 특정 기능을 사용할 수 있도록 제공되는 함수와 사용방법에 대한 약속
- 대표적인 그래픽스 API: OpenGL, Direct3D, Vulkan 등

OpenGL 특징

- **OpenGL의 특징**

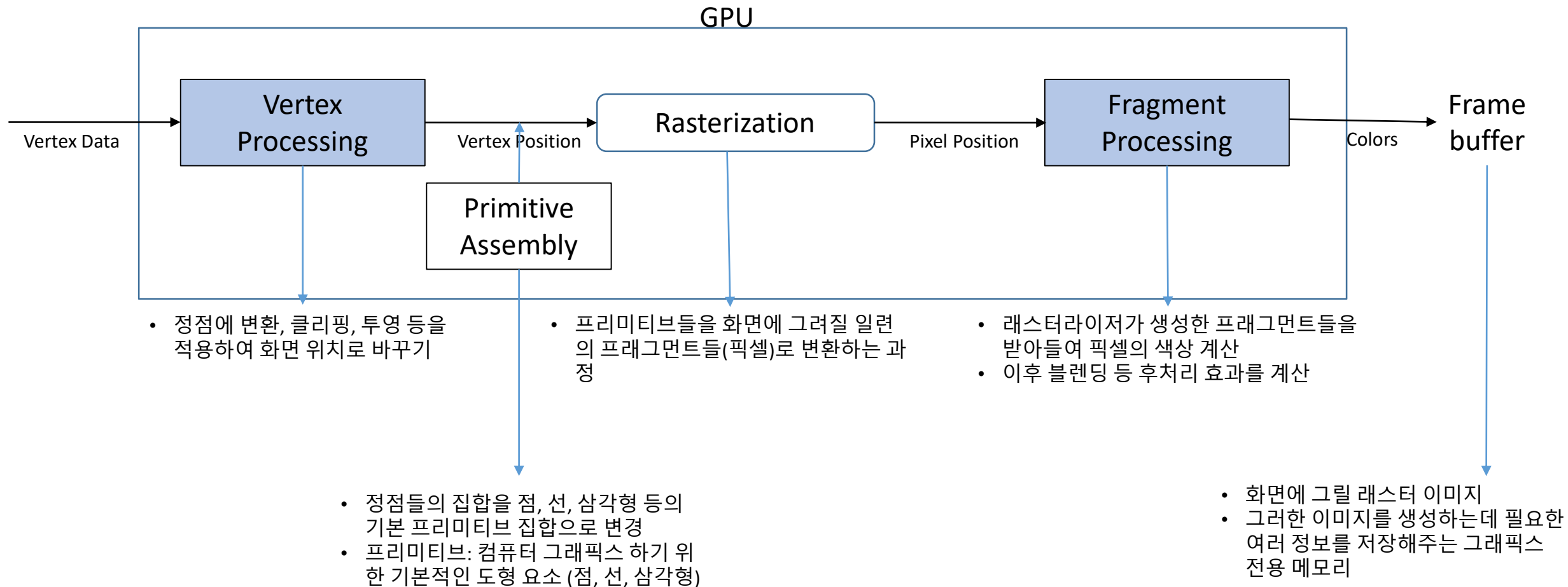
- 그래픽스 API
 - GPU를 이용해 2차원/3차원 그래픽스를 렌더링하는 Application Programming Interface
- 플랫폼에 독립적
 - Windows, macOS, Linux, Android 등 다양한 운영체제 및 호스트 언어를 지원
 - 윈도우 생성, 입력 처리 등은 별도의 라이브러리를 함께 사용
- 하드웨어 가속 지원
 - GPU를 활용한 고속 렌더링 가능
 - 셰이더 언어 (GLSL) 지원
- 상태 기반 (State Machine) 구조
 - 내부적으로 상태 머신으로 동작
 - GPU 메모리에 상태를 설정해두면 다른 상태로 바꾸기 전까지 모든 도형에 이 규칙이 계속 적용되는 방식
 - 렌더링 방식이나 설정을 변경할 때 상태를 업데이트
- 다양한 그래픽스 기능의 지원으로 응용 소프트웨어 개발이 용이
 - 기본적인 2D 및 3D 그래픽스 함수에서부터 고급 기능까지 지원
 - 셰이딩, 안티알리아싱, 텍스처 매핑, NURBS, 안개, 알파 블렌딩 등

OpenGL 버전

세대 구분	버전	출시일	주요 특징 및 도입 기술
초기 레거시 (고정 파이프라인)	1.0	1992년 6월	- 초기 버전에 따른 고정 기능 파이프라인 도입 - 디스플레이 리스트 및 기본 변환 기능
프로그래머블 (셰이더 도입)	2.0	2004년 9월	- GLSL 1.10(OpenGL Shading Language) 정식 도입 - 프로그래머블 정점/프래그먼트 셰이더 지원 - Multiple Render Targets (MRT)
모던 OpenGL (파이프라인 개편)	3.0	2008년 8월	- 고정 기능 파이프라인 폐기(Deprecated) 시작 - FBO(Framebuffer Object), VAO, Texture array 도입
	3.1	2009년 3월	레거시 기능 본격 제거, 인스턴싱(Instancing), UBO 지원
	3.2	2009년 8월	- Core/Compatibility 컨텍스트 분리 - 지오메트리 셰이더(Geometry Shader) 추가
	3.3	2010년 3월	- GLSL 버전을 OpenGL 버전과 통일(GLSL 3.30) - OpenGL 4.0 기능과의 호환성 제공
고성능 / 최신 (테셀레이션 & GPGPU)	4.0	2010년 3월	- 테셀레이션 셰이더(Tessellation Shader) 도입 64비트 정밀도 double 타입 셰이더 연산 지원
	4.3	2012년 8월	- 컴퓨트 셰이더(Compute Shader) 도입 (GPU 범용 연산) - SSBO(Shader Storage Buffer Object) 지원
	4.6	2017년 7월	- 최종 메이저 버전 - Vulkan 연동

그래픽스 파이프라인

- 그래픽스 파이프라인: 공간에 존재하는 물체의 데이터를 받아 화면에 보여주기 위한 이미지 데이터를 만들어가는 처리 과정



그래픽스 파이프라인

- **고정 파이프라인 (Fixed-function pipeline)**

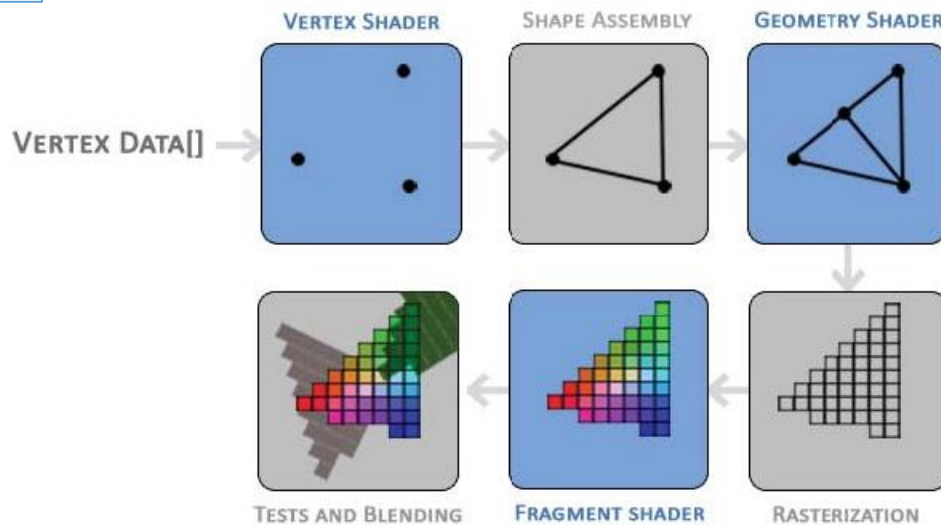
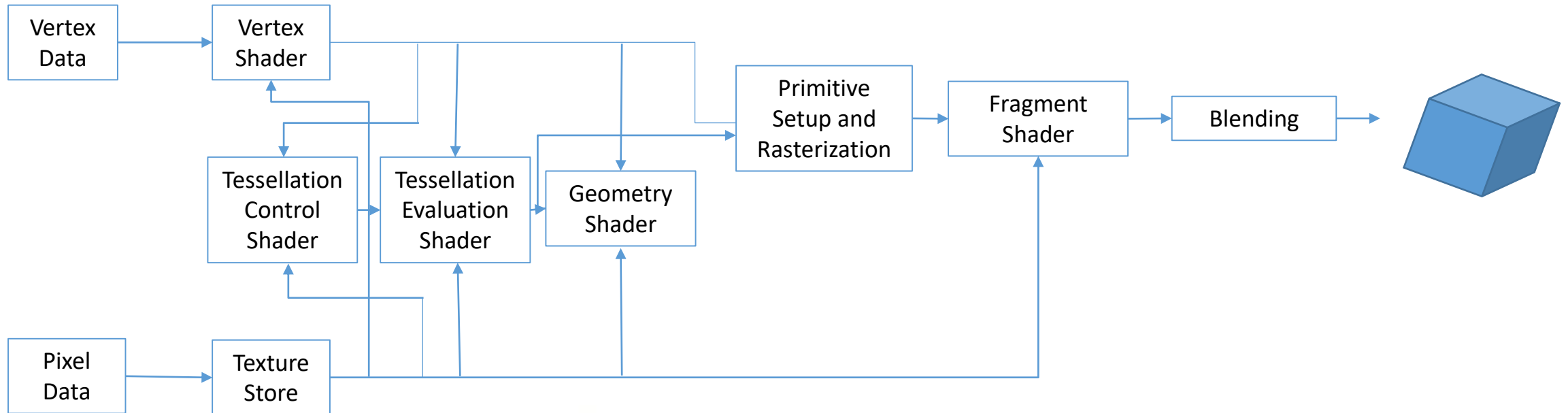
- 그래픽스 하드웨어(GPU) 내부의 모든 연산 과정과 알고리즘이 칩 레벨에서 하드웨어로 고정(Hard-wired)되어 있던 방식으로 1990년대부터 2000년대 초반까지의 OpenGL(1.x 버전)이 이 방식
- 특징:
 - 수정 불가: 하드웨어에 정해진 기능과 순서대로만 동작
 - 사용 편함: 코드가 단순
 - 효과 제한: 정해진 공식 외의 새로운 그래픽 효과를 만들기 어렵다
- 코드 흐름: 행렬 설정 → glBegin() → 위치/색상 지정 → glEnd() → 화면 출력

- **프로그래머블 파이프라인 (Programmable-function pipeline)**

- 프로그래머블 파이프라인은 GPU의 특정 하드웨어 블록을 개발자가 직접 작성한 프로그램(셰이더)으로 대체하여 실행할 수 있도록 개방한 방식으로 2000년대 초반(OpenGL 2.0 이상) 도입되어 현대 모든 GPU 연산의 표준
- 특징:
 - 코드 작성: 셰이더(Vertex, Pixel 등)를 직접 짜서 GPU 동작을 제어
 - 자유로움: 빛, 그림자, 특수 효과 등 원하는 모든 그래픽 표현이 가능
 - 배우기 어려움: 구조가 복잡
- 코드 흐름: 셰이더 컴파일/링크 → GPU 버퍼에 데이터 전송 → 셰이더 프로그램 바인딩 → glDrawArrays () 호출

그래픽스 파이프라인

- Programmable-function pipeline



출처: Programmable vs. Fixed-Function Pipeline in Real-Time Computer Graphics

[Mladen Korunoski](#), [Marjan Gushev](#), [Vladimir Zdraveski](#)

Published 1 July 2019

Computer Science

IEEE EUROCON 2019 -18th International Conference on Smart Technologies

GPU와 Shader

- **GPU (Graphics Processing Unit)**

- 설계 특징:

- CPU보다 성능은 낮지만 아주 단순한 연산이 가능한 **수천 개의 코어**(보통 수천 개 단위)로 구성되어 있다.
 - 복잡한 알고리즘을 처리하기보다는, 동시에 쏟아지는 대규모 데이터를 한 번에 담아 연산하도록 설계되었다.

- 작동 방식: 병렬 처리 - Parallel Processing

- 수천 개의 데이터를 각 코어에 나누어 준 뒤, **동시에 일제히(Parallel)** 연산

- 주요 역할:

- 3D 화면을 그리기 위한 수백만 개의 정점(Vertex) 좌표 변환
 - 모니터 화면에 출력될 수백만 개의 픽셀(Pixel) 색상 연산 및 조명(Shading) 계산
 - 딥러닝/인공지능 모델 학습, 영상 인코딩, 암호 화폐 채굴 등 대규모 수학 행렬 연산

- GPU에서 셰이더 (Shader)란

- GPU 내부의 수천개 코어에서 직접 실행되는 작고 정교한 미니 프로그램

- 셰이더 작동 방법

- C++ (CPU) 코드 작성: 개발자가 텍스트 파일이나 문자열 형태로 셰이더 코드(GLSL, HLSL 등)를 작성
 - GPU에 전송 및 컴파일: 실행 시점에 C++ 프로그램이 이 텍스트 코드를 GPU로 보내어 GPU 전용 기계어로 즉석 컴파일
 - 병렬 연산 실행: GPU 내부의 수천 개 코어가 정점(Vertex) 및 픽셀(Pixel) 데이터마다 이 셰이더 프로그램을 일제히 동시에 실행

GPU와 Shader

- 핵심 셰이더

- 1. 버텍스 셰이더 (Vertex Shader)

- 담당 영역: 3D 모델을 구성하는 정점(Vertex) 하나하나.
 - 주요 역할:
 - 3D 좌표(x, y, z)를 모니터 화면용 좌표(MVP 행렬 곱셈 연산)로 변환
 - 캐릭터가 움직일 때 뼈대(Bone)에 맞춰 정점 위치를 변환
 - 특징: 정점의 개수만큼 독립적으로 연산이 실행된다.

- 2. 프래그먼트 셰이더 (Fragment Shader / Pixel Shader)

- 담당 영역: 화면에 그려질 각 픽셀 후보(Fragment) 하나하나.
 - 주요 역할:
 - 픽셀의 최종 색상(RGB)과 투명도(Alpha)를 결정
 - 텍스처(이미지)를 입히고, 빛과 그림자 공식(Phong, PBR 조명 모델)을 연산하며, 물의 반사나 카툰 셰이딩 같은 특수 효과를 구현
 - 특징: 화면 해상도에 비례하여 수백만 번 병렬로 동시 연산된다.

- Legacy OpenGL은 OpenGL이 GPU 처리과정을 상당부분 정해주었지만,
 - Modern OpenGL에서는 셰이더를 통해 프로그래머가 GPU의 일부 처리를 직접 작성한다.

OpenGL 라이브러리

- 오픈지엘 라이브러리

- GL (OpenGL Core Library):

- 렌더링 기능을 제공하는 기본적인 함수들의 집합

- GLU (OpenGL Utility Library):

- GL 함수로 작성되어 있는 고급 기능을 제공하는 함수들의 집합

- 마이크로소프트 윈도우 운영체제에서 동작하는 비주얼 스튜디오는 OpenGL1.1까지만 지원

- Windows의 opengl32.dll은 OpenGL 1.1 함수만 기본 제공
 - glGenBuffers, glCreateShader 같은 최신 함수는 헤더에 선언조차 없음

- 최신 함수의 실제 주소는 GPU 드라이버 안에 있고, 실행 중 직접 찾아와야 함

- OpenGL 확장 함수 로더로 해당 라이브러리를 이용하여 OpenGL2.0 이상의 버전을 사용할 수 있다.
 - GLEW가 이 과정을 자동화

- **GLEW (GL Extention Wrangler):** OpenGL 축 기능이나 최신 OpenGL 함수들을 프로그램에서 사용할 수 있도록 연결해주는 역할

- 현재 OpenGL Context 에서 사용할 수 있는 OpenGL함수들을 찾아서 사용할 수 있도록 GLEW를 초기화한다.
 - 사용 방법) 헤더파일 추가 후 초기화 함수 호출

```
#include <glew.h>
```

```
glewInit();
```

- 핵심 규칙: glewInit()은 반드시 OpenGL Context 생성 이후에 호출

OpenGL 라이브러리

- 윈도우, 사용자 상호작용 지원 라이브러리

- GLUT (OpenGL Utility Toolkit): 윈도우를 생성, 사용자와의 상호작용 처리 함수들을 지원
- freeGLUT: 더 이상 개선되지 않는 GLUT의 대체 라이브러리로 윈도우를 다루는 함수들 지원
- GLAUX: 오픈지엘 보조 라이브러리
- GLFW (GL Frame Work): 오픈지엘을 위한 오픈소스 멀티-플랫폼 라이브러리
 - GLUT와 같은 역할을 하는 라이브러리로 윈도우 생성, 상호작용 처리 함수들을 지원
- **GLFW (Graphics Library Framework):** 크로스 플랫폼 윈도우, 입력 라이브러리
 - 운영체제에 맞는 윈도우(창) 생성
 - OpenGL 컨텍스트 생성 및 관리
 - 키보드 · 마우스 · 조이스틱 입력 이벤트 처리
 - 타이머(시간 측정) 제공 → 애니메이션의 핵심 재료
 - API의 WNDCLASS · CreateWindow · WndProc 메시지 루프를, 플랫폼 독립적으로 감싼 것이 GLFW

- 수학을 위한 라이브러리

- **GL Mathematics (GLM)**
 - 그래픽스 수학을 C++에서 쉽게 사용할 수 있도록 제공하는 header-only 수학 라이브러리
 - 벡터와 행렬 연산 및 그래픽스에 필요한 다양한 수학 함수를 제공
 - 카메라와 투영 변환을 쉽게 만들 수 있음
 - GLSL과 매우 비슷한 문법을 이용

OpenGL 라이브러리

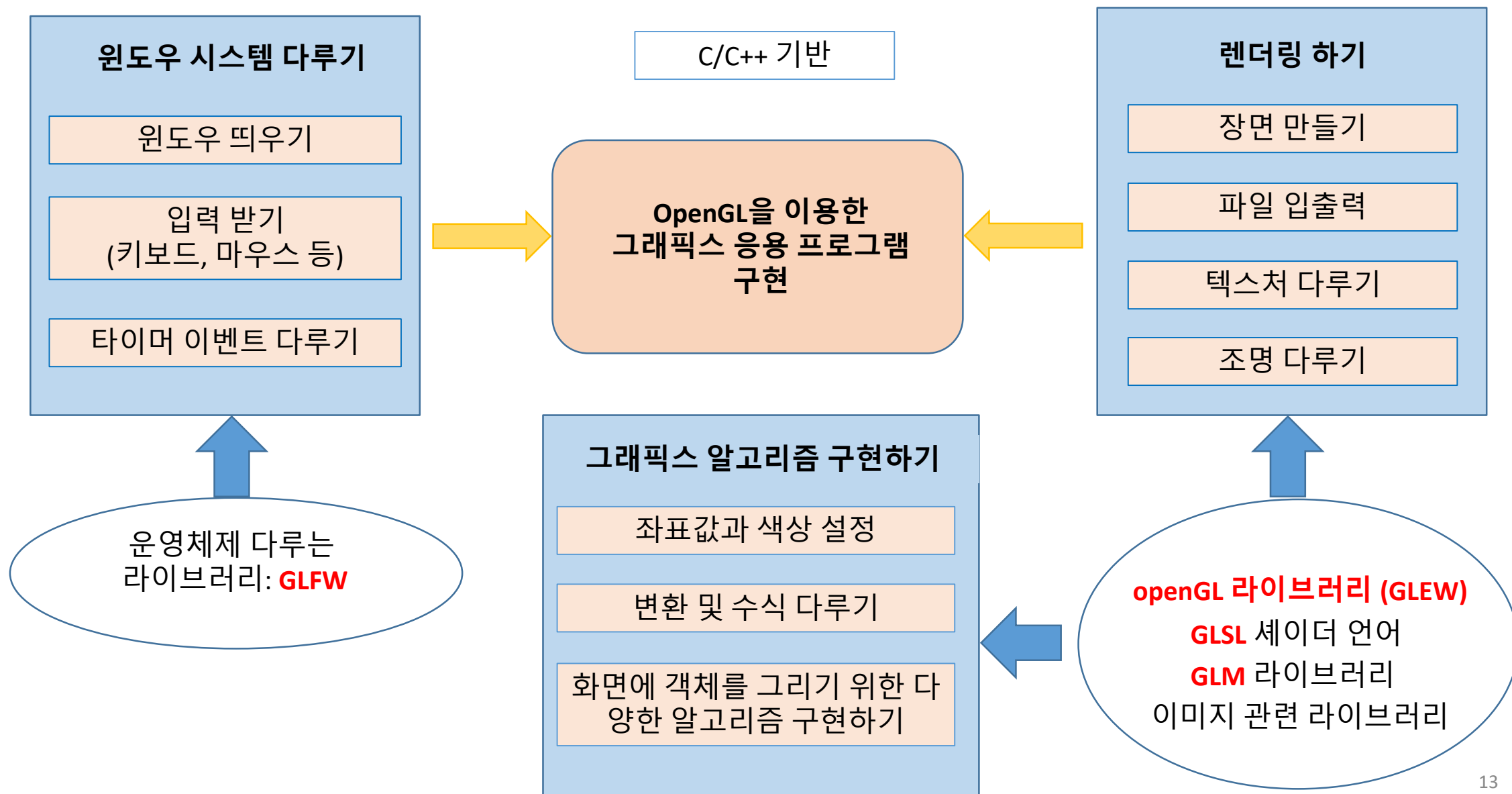
- 셰이더 작성을 위한 언어

- **OpenGL Shading Language (GLSL)**

- OpenGL의 셰이더를 컨트롤할 수 있는 C언어와 유사한 형태의 언어
 - GLSL은
 - OpenGL에서 사용할 수 있는 그래픽스 셰이딩 언어 (Graphics Library Shading Language)
 - 특징:
 - C언어 기반의 문법
 - GPU 병렬 처리에 최적화된 내장 타입: 벡터, 행렬 타입이 언어 자체의 기본 데이터타입으로 내장되어 있음
 - 벡터 사용을 위한 특별한 연산 지원: 스윙클링 연산
 - 다양한 내장 수학 함수
 - GLSL코드는 메인 소스 코드 내에 문자열 형태로 포함되어 실행 시점에 GPU로 전달되어 컴파일 및 link 된다.
 - 즉, GLSL을 사용하여 셰이더 코드는 메인 프로그램과는 분리되게 구현한다.
 - 구현된 코드는 GPU를 통해 렌더링이 수행된다.

구성 요소	역할
OpenGL	그래픽 렌더링 API
GLFW	윈도우, 입력, OpenGL Context
GLEW/GLAD	OpenGL 함수 로딩
GLM	벡터·행렬·변환 계산
GLSL	GPU에서 실행되는 셰이더 언어

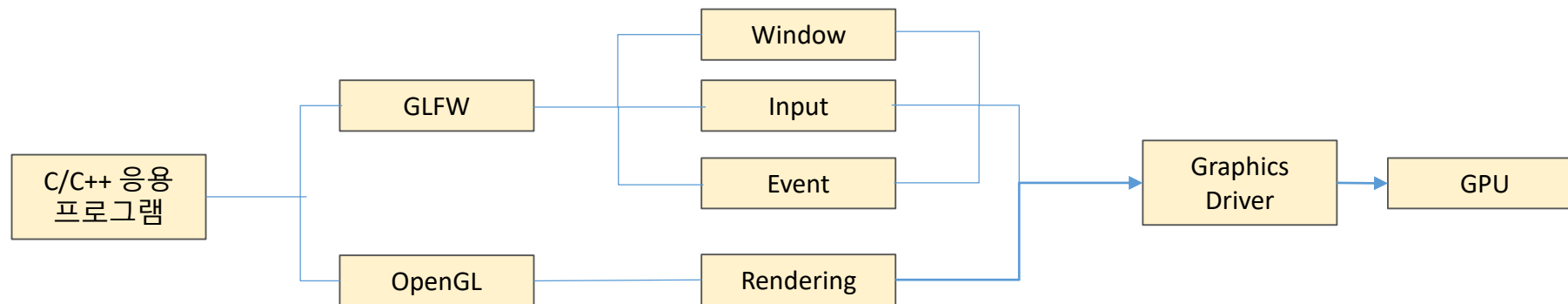
프로그램 구성



1. GLFW (GL FrameWork)

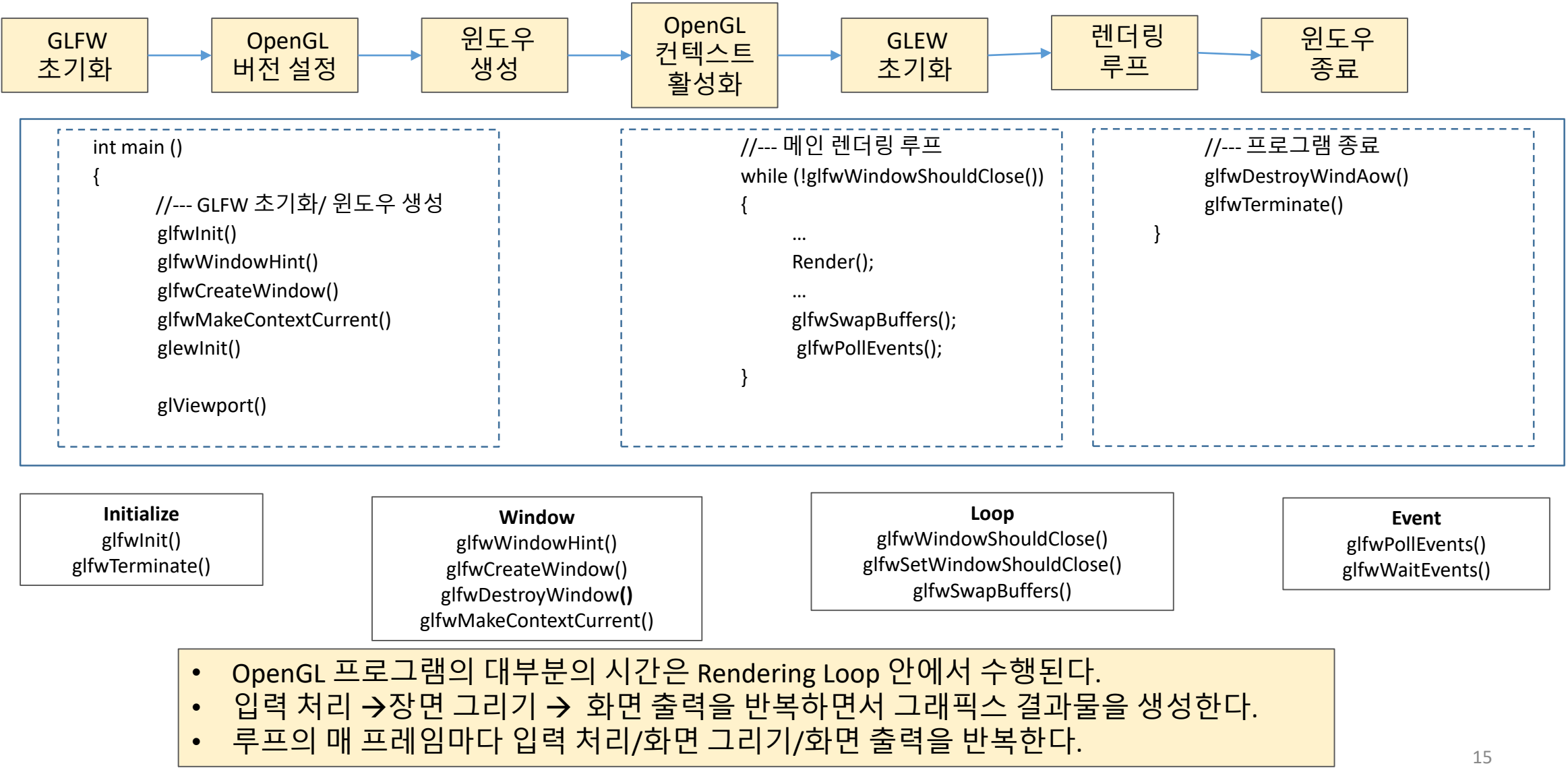
- **GLFW (GL Frame Work)**

- OpenGL 프로그램을 실행하기 위한 윈도우와 OpenGL Context를 만들고, 키보드.마우스 입력과 이벤트 등을 처리할 수 있도록 해주는 크로스 플랫폼 라이브러리
 - OpenGL은 화면에 그림을 그리는 기능만 제공
- GLFW는
 - 윈도우 생성 및 관리
 - OpenGL context 생성 및 관리
 - 키보드/마우스 입력 등 사용자 입력과 이벤트 처리, 시간 등의 기능 수행
 - 사용자 입력과 이벤트 처리는 폴링(Polling) 방식 또는 콜백 (Callback) 방식을 사용
 - 폴링 방식: 내가 계속 물어본다
 - 콜백 방식: 이벤트가 발생하면 GLFW가 나를 호출한다.



GLFW (GL FrameWork)

• GLFW 프로그램 흐름도



GLFW (GL FrameWork)

- **컨텍스트(Context):**

- 오픈지엘이 동작하기 위해 필요한 모든 상태와 리소스를 담고 있는 실행 환경
- 오픈지엘의 여러 상태와 객체를 관리하기 위한 정보가 저장
 - 현재 사용 중인 Shader Program
 - 현재 바인딩된 VAO, 현재 바인딩된 VBO
 - Texture 상태 / Blend 상태 / Depth Test 상태 / Viewport 상태
 - OpenGL 버전 및 기능
 - OpenGL 객체들의 관리 정보
- 컨텍스트가 생성되면 오픈지엘을 사용할 수 있고, 컨텍스트가 파괴되면 해당 오픈지엘 환경도 삭제됨
- 컨텍스트는 오픈지엘이 그릴 수 있는 작업 공간으로 창을 만들고 활성화 해야만 함수들을 정상적으로 동작
 - GLFW로 윈도우를 만들면 운영체제가 그 윈도우에 해당하는 오픈지엘 컨텍스트를 준비
 - **glfwMakeContextCurrent** (window) 함수 호출로 해당 컨텍스트를 현재 스레드에 바인딩
 - 오픈지엘은 내부적으로 컨텍스트에 모든 상태 (현재 색상, 뷰포트 크기, 바인딩된 버퍼, 활성 셰이더 등)를 저장 → 오픈지엘 함수들을 호출하면 그 값들이 현재 컨텍스트의 상태 저장소에 기록된다.
- **GLFW로 윈도우를 만들면 오픈지엘 컨텍스트가 자동으로 따라오고 우리는 그걸 활성화해서 사용한다.**

- Window: 그림을 보여주는 캔버스
- OpenGL Context: 그림을 그리는 작업 공간과 상태 정보

윈도우 띄우기

```
#include <GL/glew.h>
#include <GL/glfw3.h>
#include <iostream>
```

```
int main() {
    //--- GLFW 초기화
    if (!glfwInit()) {
        std::cerr << "GLFW 초기화 실패!" << std::endl;
        return -1;
    }
```

//--- OpenGL 버전 설정 (예: 3.3 Core Profile)

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

//--- 윈도우 생성

```
GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL Window", nullptr, nullptr);
if (!window) {
    std::cerr << "윈도우 생성 실패!" << std::endl;
    glfwTerminate();
    return -1;
}
```

//--- 컨텍스트 설정

```
glfwMakeContextCurrent(window);
```

//--- GLEW 초기화

```
glewExperimental = GL_TRUE; // 최신 기능 사용
if (glewInit() != GLEW_OK) {
    std::cerr << "GLEW 초기화 실패!" << std::endl;
    return -1;
}
```

//--- 뷰포트 설정

```
glViewport(0, 0, 800, 600);
```

//--- 메인 루프

```
while (!glfwWindowShouldClose(window)) {
    // 입력 처리
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);

    // 화면 지우기 (파란색)
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f); // RGBA (파랑)
    glClear(GL_COLOR_BUFFER_BIT);

    // 버퍼 교체
    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

//--- 종료 처리

```
glfwDestroyWindow(window);
glfwTerminate();
return 0;
}
```

함수들

• 라이브러리 초기화/종료

- void **glfwInit** ();
 - GLFW 라이브러리를 사용하기 전 내부 시스템을 초기화
 - 리턴 값: GLFW_TRUE / GLFW_FALSE
- void **glfwTerminate** ();
 - 라이브러리 종료

• 윈도우 생성/삭제

- GLFWwindow* **glfwCreateWindow** (int width, int height, const char* title, GLFWmonitor* monitor, GLFWwindow* share);
 - 윈도우를 생성
 - width/height: 윈도우의 폭/높이
 - title: 윈도우의 제목을 나타내는 문자열
 - monitor: 전체 화면이면 monitor, 일반 창이면 NULL
 - share: 오픈지엘 컨텍스트 공유용, 대개 NULL
 - 리턴 값: 생성된 윈도우를 가리키는 포인터 값, 실패하면 NULL
 - 사용 예) 800 x600 크기의 윈도우 만들기: GLFWwindow* window = glfwCreateWindow (800, 600, "OpenGL", nullptr, nullptr);
 - 사용 예) 전체 화면: GLFWwindow* window = glfwCreateWindow (1920, 1080, "Game", glfwGetPrimaryMonitor(), nullptr);
 - 전체 화면인 경우의 width와 height 는 화면에 사용할 해상도를 의미한다.
 - glfwGetPrimaryMonitor() 함수는 전체화면으로 사용할 주 모니터를 얻는 함수
- void **glfwDestroyWindow** (GLFWwindow* window);
 - 생성한 윈도우 객체를 삭제

함수들

- 윈도우를 생성하기전에 생성될 윈도우와 오픈지엘 컨텍스트의 속성 지정 함수
 - void **glfwWindowHint** (int hint, int value);
 - 윈도우나 오픈지엘 컨텍스트의 속성을 변경하고 싶을 때만 사용하는 선택적 설정함수
 - hint: 설정하려는 속성
 - value: 해당 속성의 값
 - 사용 예) 오픈지엘 버전 3.3의 modern 오픈지엘로 지정

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

Hint	값의 예	의미	기본 값
GLFW_CONTEXT_VERSION_MAJOR	3, 4	OpenGL 주 버전	0
GLFW_CONTEXT_VERSION_MINOR	0~6	OpenGL 부 버전	0
GLFW_OPENGL_PROFILE	GLFW_OPENGL_CORE_PROFILE/ GLFW_OPENGL_COMPAT_PROFILE	프로파일 버전 CORE_PROFILE: modern openGL COMPAT_PROFILE: legacy openGL	GLFW_OPENGL_ANY_PROFILE
GLFW_OPENGL_FORWARD_COMPAT	GLFW_TRUE	Forward Compatible Context	GLFW_FALSE
GLFW_RESIZABLE	GLFW_TRUE/FALSE	창 크기 변경 가능 여부	GLFW_TRUE
GLFW_VISIBLE	GLFW_TRUE/FALSE	초기 표시 여부	GLFW_TRUE
GLFW_DECORATED	GLFW_TRUE/FALSE	제목 표시줄 및 테두리 표시 여부	GLFW_TRUE
GLFW_FLOATING	GLFW_TRUE/FALSE	항상 위(Topmost) 창	GLFW_FALSE
GLFW_MAXIMIZED	GLFW_TRUE/FALSE	최대화 상태로 시작	GLFW_FALSE
GLFW_DOUBLEBUFFER	GLFW_TRUE/FALSE	더블 버퍼 사용	GLFW_TRUE
GLFW_DEPTH_BITS	16, 24, 32	깊이 버퍼 비트 수 (디폴트 값: 24)	24

함수들

- 현재 컨텍스트 지정

- void **glfwMakeContextCurrent** (GLFWwindow* window);

- 지정한 윈도우의 OpenGL Context를 현재 스레드에 연결하는 함수.

- window: glfwCreateWindow() 함수가 반환한 윈도우 포인터. 이 윈도우의 OpenGL Context를 활성화

- 윈도우를 생성하고 반드시 이 함수를 호출해야 한다. 이후에 GLEW 초기화 함수를 호출한다.

- 사용 순서: glfwInit(); → glfwCreateWindow() → glfwMakeContextCurrent() → glewInit() → OpenGL 함수 사용

- 사용 예)

```
int main () {  
    glfwInit();
```

```
    GLFWwindow* window = glfwCreateWindow (800,600,"OpenGL",nullptr,nullptr);
```

```
    glfwMakeContextCurrent (window);
```

```
    glewInit();
```

```
    glClearColor(...);
```

```
    glViewport(...);
```

```
    ...
```

```
}
```

함수들

• 프로그램 종료 여부 확인

- int **glfwWindowShouldClose** (GLFWwindow* window);
 - 생성된 윈도우를 닫아야 하는 상황인지 여부를 확인하거나 닫기 상태를 직접 제어할 때 사용
 - window: 종료 여부를 확인할 GLFW 윈도우 객체
 - **GLFW 프로그램의 메인 루프를 제어하는 핵심 함수로** 아래와 같은 형태로 사용된다.

```
while (!glfwWindowShouldClose(window))  
{  
    //--- 화면 그리기  
    glfwSwapBuffers(window);  
    glfwPollEvents();  
}
```

- 반환값: GLFW_TRUE/GLFW_FALSE
 - GLFW_FALSE (0): 윈도우를 계속 실행
 - GLFW_TRUE (1): 윈도우를 종료
 - 사용자가 윈도우 우측 상단의 'x (닫기)' 버튼을 클릭했을 때
 - OS 창 관리자에서 **Alt + F4** 등의 창 닫기 단축키를 눌렀을 때
 - 시스템 관리자나 타 프로세스에서 해당 창에 Close 이벤트를 보냈을 때

• 윈도우 종료 플래그 설정

- void **glfwSetWindowShouldClose** (GLFWwindow* window, int value);
 - 특정 윈도우의 닫기 상태를 코드로 직접 설정할 때 사용
 - 반환값: GLFW_TRUE/GLFW_FALSE

함수들

- 사용 예) 에러 체크 생략되어 있음

```
//--- 메인 함수
int main() {
    glfwInit();
    GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL Window", nullptr, nullptr);
    glfwMakeContextCurrent(window);
    glewExperimental = GL_TRUE;

    //--- 메인 렌더링 루프: glfwWindowShouldClose가 GL_TRUE(1)를 반환하면 루프탈출
    while (!glfwWindowShouldClose(window))
    {
        //--- 이벤트 처리
        glfwPollEvents();

        //--- 입력 처리: 키보드/마우스 이벤트 처리
        processInput(window);

        //--- 렌더링 연산: OpenGL 그리기 명령들
        glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);

        //--- 버퍼 교체
        glfwSwapBuffers(window);
    }

    glfwDestroyWindow(window);
    glfwTerminate();
    return 0;
}
```

함수들

- 사용 예) 수동으로 닫기 조작하기: 앞의 processInput 함수에서 ESC 키를 눌러 윈도우를 닫는 형태

```
void processInput (GLFWwindow* window)
{
    //--- 사용자가 ESC 키를 누르면 윈도우 닫기 상태를 GL_TRUE로 강제 변경
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
    {
        glfwSetWindowShouldClose (window, GL_TRUE);           //--- 다음 루프 조건문에서 while 탈출함
    }
}
```

함수들

- 이벤트 처리

- void **glfwPollEvents** (void);
 - 운영체제(OS)의 이벤트 큐(Event Queue)에 쌓여 있는 이벤트(키보드 입력, 마우스 이동/클릭, 창 크기 변경, 창 이동 등)를 가져와 처리하는 함수
 - 이 함수가 호출되는 순간, OS로부터 들어온 최신 이벤트들을 읽어 들여 GLFW 내부 상태를 갱신하고, 개발자가 등록해 둔 각종 **콜백 함수(Callback Functions)** (예, glfwSetKeyCallback, glfwSetCursorPosCallback 등) 를 호출
 - 메인 루프에서 매 프레임 호출해야 한다. 메인 루프의 끝 부분에 배치한다.

- 이벤트가 발생할 때까지 대기

- void **glfwWaitEvents** (void);
- glfwPollEvents 함수는 이벤트를 확인하고 즉시 반환
 - 실시간 렌더링에 적합, 애니메이션 가능, 게임에서 주로 사용
 - CPU 사용량이 상대적으로 높음
- glfwWaitEvents 함수는 이벤트가 발생할 때까지 대기
 - 이벤트가 없으면 프로그램은 대기, 따라서 CPU 사용량이 낮음
 - UI 프로그램에 적합

함수들

- 더블 버퍼 교환

- void **glfwSwapBuffers** (GLFWwindow* window);
 - 더블 버퍼링 기법을 사용할 때 화면 버퍼를 교체하여 화면에 출력해주는 함수
 - 화면에 그림을 그리면 먼저 Back buffer에 그려지고 glfwSwapBuffers 함수를 호출하여 Front buffer와 교환되어 사용자가 화면에서 봄
 - 모든 렌더링 명령이 끝난 직후 메인 루프의 마지막 부분에서 호출
 - window: 버퍼를 교환할 윈도우

- 오류 확인

- const char* **glfwGetError** (const char** description);

- 윈도우 타이틀 변경

- void **glfwSetWindowTitle** (GLFWwindow* window, const char* title);

GLFW (GL FrameWork)에서 사용자 입력 처리 방식

- GLFW에서 사용자 입력 (키보드, 마우스 등)이나 윈도우 이벤트 처리 방식
 - 폴링(Polling) 방식 (상태 조회 방식): 프로그램이 반복적으로 현재 상태를 직접 확인하는 방식
 - 메인 루프 안에서 매 프레임마다 상태를 확인 → "현재 상태 기반 명령어" 처리에 적합
 - 콜백(Callback) 방식 (이벤트 처리 방식): 이벤트가 발생하면 GLFW가 자동으로 사용자 함수를 호출
 - 이벤트 발생 순간에만 처리 → "명령어 입력 순간" 을 잡아내는 데 적합

구분	폴링 방식 (Polling)	콜백 방식 (Callback)
작동 원리	개발자가 매 프레임마다 상태를 직접 체크	이벤트 발생 시 GLFW가 등록된 함수를 자동 호출
적합한 용도	• 카메라 이동 • 캐릭터 연속 이동 • 실시간 키 입력	• 단축키 및 메뉴 조작 • 마우스 클릭 처리 • UI 클릭 / 텍스트 입력 • 창 크기 변경 대응
주요 대표 함수	<code>glfwGetKey()</code> <code>glfwGetCursorPos()</code>	<code>glfwSetKeyCallback()</code> <code>glfwSetScrollCallback()</code>
상태 감지 시점	프레임 렌더링 시점	이벤트 발생 시점 (<code>glfwPollEvents()</code> 실행 중)

- 두 방식 모두 메인 루프에서 **`glfwPollEvent()` 함수를 호출**해서 운영체제(OS)의 이벤트를 GLFW 내부로 가져와야 한다.

GLFW (GL FrameWork)에서 사용자 입력 처리 방식

- 폴링/콜백 방식의 주요 함수들

이벤트	Polling 함수	설명	Callback 함수
키보드	glfwGetKey()	특정 키의 현재 상태 조회	glfwSetKeyCallback()
문자 입력		문자 입력	glfwSetCharCallback()
마우스 버튼	glfwGetMouseButton()	마우스 버튼 상태 조회	glfwSetMouseButtonCallback()
마우스 위치	glfwGetCursorPos()	현재 마우스 좌표 조회	glfwSetCursorPosCallback()
윈도우 크기	glfwGetWindowSize()	현재 창 크기 조회	glfwSetWindowSizeCallback()
Framebuffer 크기	glfwGetFramebufferSize()	현재 Framebuffer 크기 조회	glfwSetFramebufferSizeCallback()
윈도우 위치	glfwGetWindowPos()	현재 창 위치 조회	glfwSetWindowPosCallback()
창 닫기 여부	glfwWindowShouldClose()	종료 플래그 조회	glfwSetWindowCloseCallback()

함수들: 폴링 방식

- 키 입력 상태 확인

- `int glfwGetKey (GLFWwindow* window, int key);`

- 현재 키가 눌러 있는지 확인
- window: 키 상태를 확인할 윈도우
- key: 확인할 키
 - 알파벳 키: `GLFW_KEY_A`, `GLFW_KEY_B`, ... `GLFW_KEY_Z`
 - 숫자 키: `GLFW_KEY_0`, `GLFW_KEY_1`, ... `GLFW_KEY_9`
 - 기능키: `GLFW_KEY_F1`, `GLFW_KEY_F2`... `GLFW_KEY_F25`
 - 방향키: `GLFW_KEY_UP`, `GLFW_KEY_DOWN`, `GLFW_KEY_LEFT`, `GLFW_KEY_RIGHT`
 - 특수키: `GLFW_KEY_ESCAPE`, `GLFW_KEY_ENTER`, `GLFW_KEY_SPACE`, `GLFW_KEY_LEFT_SHIFT`, `GLFW_KEY_LEFT_CONTROL`, `GLFW_KEY_LEFT_ALT`
- 리턴값: `GLFW_PRESS` / `GLFW_RELEASE`
- 동작 원리: 사용자 키 입력 → 운영체제 → `glfwPollEvents()` → GLFW 내부 키 상태 → `glfwGetKey ()` → `GLFW_PRESS`
- 게임에서 가장 많이 사용, 매 프레임 상태를 확인, 좌우상하 이동에 적합

- 사용 예)

```
if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
{
    glfwSetWindowShouldClose(window, GLFW_TRUE);
}
```

함수들: 폴링 방식

- 사용 예)

```
int main()
{
    //--- GLFW 초기화
    if (!glfwInit())

    //--- OpenGL 버전 설정
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_COMPAT_PROFILE);

    //--- 윈도우 생성
    GLFWwindow* window = glfwCreateWindow(800, 600, "Keyboard Input", nullptr, nullptr);

    //--- 컨텍스트 활성화
    glfwMakeContextCurrent(window);

    //--- GLEW 초기화
    glewExperimental = GL_TRUE;
    glewInit ();

    //--- 뷰포트 설정
    glViewport(0, 0, 800, 600);

    //--- 메인 루프
    while (!glfwWindowShouldClose(window)) {

        //--- 키보드 입력 처리 함수
        inputProcess(window);
    }
```

```
//--- 화면 그리기
    glClearColor(rColor, gColor, bColor, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glfwSwapBuffers(window);
    glfwPollEvents();
}

glfwDestroyWindow(window);
glfwTerminate();

return 0;
}
//--- End of Main function

//--- 키보드 입력 처리 함수
void inputProcess(GLFWwindow* window)
{
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
    {
        //--- esc 키를 눌렀을 때 작업
    }

    if (glfwGetKey(window, GLFW_KEY_R) == GLFW_PRESS)
    {
        //--- r을 눌렀을 때 작업
    }
}
```

함수들: 폴링 방식

- 버튼의 상태 확인

- int **glfwGetMouseButton** (GLFWwindow* window, int button);

- 마우스 버튼의 상태를 조회
- window: 마우스 상태를 확인할 윈도우
- button: 확인할 마우스 버튼을 지정
 - GLFW_MOUSE_BUTTON_LEFT/ GLFW_MOUSE_BUTTON_RIGHT/ GLFW_MOUSE_BUTTON_MIDDLE
 - (=GLFW_MOUSE_BUTTON_1/ GLFW_MOUSE_BUTTON_2/ GLFW_MOUSE_BUTTON_3)
- 리턴 값: GLFW_PRESS/GLFW_RELEASE

- 사용 예) 왼쪽 버튼 상태 확인

```
if (glfwGetMouseButton(window, GLFW_MOUSE_BUTTON_LEFT) == GLFW_PRESS)
{
    std::cout << "Left Button Press\n";
}
```

- 사용 예) 오른쪽 버튼 상태 확인

```
if (glfwGetMouseButton (window, GLFW_MOUSE_BUTTON_RIGHT) == GLFW_PRESS)
{
    std::cout << "Right Button Press\n";
}
```

함수들: 폴링 방식

- 마우스 위치 얻기

- void **glfwGetCursorPos** (GLFWwindow* window, double* xpos, double* ypos);

- 마우스의 현재 위치를 조회

- xpos/ypos: 현재 커서의 x 좌표/y 좌표 (윈도우 기준의 픽셀 좌표)

- 클릭 위치, 드래그 시 사용

- 사용 예) 왼쪽 버튼을 눌러 드래그 했을 때 → 절대 위치

- double x, y;

- if (glfwGetMouseButton (window, GLFW_MOUSE_BUTTON_LEFT) == GLFW_PRESS)

- {

- glfwGetCursorPos (window, &x, &y);

- std::cout << x << ", " << y << std::endl;

- }

함수들: 폴링 방식

- 사용 예) 마우스 드래그: 이전 좌표를 기억하게 한다. → 마우스 이동량 계산 → 상대 위치 얻기

```
static double oldX = 0.0;
static double oldY = 0.0;
static bool dragging = false;
```

```
double mouseX, mouseY;
```

```
if (glfwGetMouseButton(window, GLFW_MOUSE_BUTTON_LEFT) == GLFW_PRESS) //-- 왼쪽 버튼이 눌러 있는가?
{
    glfwGetCursorPos(window, &mouseX, &mouseY);          //-- 마우스 위치 가져오기

    if (!dragging) {                                     // 드래그 시작
        oldX = mouseX;
        oldY = mouseY;
        dragging = true;
    }
    else {
        double dx = mouseX - oldX;    //-- 상대 이동량
        double dy = mouseY - oldY;

        std::cout << "dx = " << dx << ", dy = " << dy << std::endl;

        // 현재 위치를 다음 프레임의 이전 위치로 저장
        oldX = mouseX;    oldY = mouseY;
    }
}
else
    dragging = false;    //-- 버튼을 놓으면 드래그 종료
```

함수들: 폴링 방식

- 윈도우 크기 얻기

- void **glfwGetWindowSize** (GLFWwindow* window, int* width, int* height);
 - 현재 윈도우의 크기 (논리적 윈도우 크기)를 얻어온다.
 - width/height: 윈도우의 가로/세로 크기

- 윈도우의 프레임 버퍼 크기 얻기

- void **glfwGetFramebufferSize** (GLFWwindow* window, int* width, int* height);
 - 실제로 렌더링하는 프레임버퍼의 크기 (픽셀 단위)
 - width/height: 프레임버퍼의 가로/세로 크기
 - glViewport 로 뷰포트를 설정할 때 사용한다.

함수	반환하는 크기	용도
glfwGetWindowSize()	논리적 윈도우 크기(Logical Size), 즉 사용자가 보는 창의 크기	UI 배치, 창 크기 확인
glfwGetFramebufferSize()	실제 픽셀 크기(Physical Pixel Size), 즉 오픈지엘이 실제로 렌더링하는 픽셀 크기	glViewport() 설정, OpenGL 렌더링

함수들: 폴링 방식

- 윈도우 위치 얻기

- void **glfwGetWindowPos** (GLFWwindow* window, int* xpos, int* ypos);
 - 윈도우의 위치를 얻는다.
 - Window: 위치를 알아낼 GLFW 윈도우
 - Xpos/ypos: 윈도우의 좌측 상단 모서리의 x좌표/y좌표

- 창 닫기 여부 확인

- int **glfwWindowShouldClose** (GLFWwindow* window);
 - 윈도우를 닫아야 하는 상태인가를 확인하는 함수

함수들: 콜백 방식

- 콜백 함수
 - 콜백 방식에서는
 - 콜백 함수를 작성하고
 - 그 사용자 정의 함수를 GLFW에 함수 등록해야 한다.

함수들: 콜백 방식

- 키보드 이벤트

- 콜백 함수 등록

- GLFWkeyfun **glfwSetKeyCallback** (GLFWwindow* window, GLFWkeyfun callback);

- callback: 키보드 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **KeyCallback** (GLFWwindow* window, int key, int scancode, int action, int mods);

- key: 키코드

- scancode: 운영체제가 사용하는 하드웨어 스캔 코드, (사용하지 않음)

- action: 키의 상태 (GLFW_PRESS (키를 처음 눌렀을 때) / GLFW_RELEASE / GLFW_REPEAT (키를 계속 누르고 있을 때 자동 반복 입력))

- mods: modifier key 상태 (GLFW_MOD_SHIFT/GLFW_MOD_CONTROL/GLFW_MOD_ALT/GLFW_MOD_SUPER)

- 사용 예)

- glfwSetKeyCallback(window, KeyCallback); //--- 콜백 함수 등록

- void KeyCallback (GLFWwindow* window, int key, int scancode, int action, int mods) //--- 등록한 사용자 정의 콜백 함수

- {

- if (action == GLFW_PRESS)

- {

- switch (key) {

- case GLFW_KEY_ESCAPE: glfwSetWindowShouldClose(window, GLFW_TRUE); break;

- case GLFW_KEY_R: ...;

- case GLFW_KEY_G:...;

- }

- }

함수들: 콜백 방식

- 사용 예)

```
int main()
{
    //--- GLFW 초기화
    if (!glfwInit())

    //--- OpenGL 버전 설정
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_COMPAT_PROFILE);

    //--- 윈도우 생성
    GLFWwindow* window = glfwCreateWindow(800, 600, "Keyboard Input", nullptr, nullptr);

    //--- 컨텍스트 활성화
    glfwMakeContextCurrent(window);

    //--- GLEW 초기화
    glewExperimental = GL_TRUE;
    glewInit ();

    //--- 뷰포트 설정
    glViewport(0, 0, 800, 600);

    //--- 키보드 콜백 함수 등록
    glfwSetKeyCallback(window, keyCallback);

    //--- 메인 루프
    while (!glfwWindowShouldClose(window)) {
```

```
//--- 화면 그리기
    glClearColor(rColor, gColor, bColor, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    glfwSwapBuffers(window);
    glfwPollEvents();
}

glfwDestroyWindow(window);
glfwTerminate();

return 0;
}
//--- End of Main function

//--- 키보드 콜백 함수
void keyCallback (GLFWwindow* window, int key, int scancode, int action, int mods)
{
    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) {
        // ESC 키를 눌렀을 때 작업
        glfwSetWindowShouldClose(window, GLFW_TRUE);
    }

    if (key == GLFW_KEY_R && action == GLFW_PRESS)
    {
        // R 키를 눌렀을 때 작업
    }
}
```

함수들: 콜백 방식

- **마우스 이벤트**

- 콜백 함수 등록

- GLFWmousebuttonfun **glfwSetMouseButtonCallback** (GLFWwindow* window, GLFWmousebuttonfun callback);
 - callback: 마우스 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **MouseButtonCallback** (GLFWwindow* window, int button, int action, int mods);
 - window: 마우스가 발생한 윈도우
 - button: 눌린 마우스 버튼 (GLFW_MOUSE_BUTTON_LEFT/GLFW_MOUSE_BUTTON_RIGHT/GLFW_MOUSE_BUTTON_MIDDLE)
 - action: GLFW_PRESS / GLFW_RELEASE
 - mods: Shift, Ctrl, Alt 등의 수정키 정보

- **사용 예)**

```
glfwSetMouseButtonCallback (window, MouseButtonCallback);           //--- 콜백 함수 등록
```

```
void MouseButtonCallback (GLFWwindow* window, int button, int action, int mods)    //--- 등록된 사용자 정의 콜백 함수
{
    if (button == GLFW_MOUSE_BUTTON_LEFT && action == GLFW_PRESS)
    {
        double x, y;
        glfwGetCursorPos(window, &x, &y); //--- 마우스의 위치를 얻어온다.

        std::cout << "Clicked : " << x << ", " << y << std::endl;
    }
}
```

함수들: 콜백 방식

- **마우스 이동 이벤트**

- 콜백 함수 등록

- GLFWcursorposfun **glfwSetCursorPosCallback** (GLFWwindow* window, GLFWcursorposfun callback);

- callback: 마우스 이동 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **CursorPosCallback** (GLFWwindow* window, double xpos, double ypos);

- window: 이벤트가 발생한 윈도우

- xpos/ypos: 현재 마우스의 x좌표/y좌표 (픽셀 단위)

- 사용 예)

```
glfwSetCursorPosCallback (window, CursorPosCallback);           //--- 콜백 함수 등록
```

```
void CursorPosCallback ( GLFWwindow* window, double xpos, double ypos)           //--- 등록된 사용자 정의 콜백 함수
{
    std::cout << xpos << ", " << ypos << std::endl;
}
```

- MouseButtonCallback 함수는 마우스 버튼을 눌렀거나 놓았을 때만 호출 → 버튼 클릭, 메뉴선택 등 클릭 이벤트 처리
 - CursorPosCallback 함수는 마우스가 움직일 때마다 호출 → 드래그, 마우스 추적 등 마우스 위치 추적

함수들: 콜백 방식

- **마우스 휠 이벤트**

- 콜백 함수 등록

- GLFWscrollfun **glfwSetScrollCallback** (GLFWwindow* window, GLFWscrollfun callback);
 - callback: 마우스 휠 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **ScrollCallback** (GLFWwindow* window, double xoffset, double yoffset);
 - window: 이벤트가 발생한 윈도우
 - xoffset/yoffset: 수평 휠 이동량 (대부분 0)/수직 휠 이동량

- **사용 예)**

```
glfwSetScrollCallback (window, ScrollCallback);           //--- 콜백 함수 등록
```

```
void ScrollCallback ( GLFWwindow* window, double xoffset, double yoffset)           //--- 등록된 사용자 정의 콜백 함수
{
    std::cout << "Scroll : " << xoffset << ", " << yoffset << std::endl;
    //--- 휠을 앞으로 한 칸 굴리면: Scroll: 0, 1
    //--- 휠을 뒤로 한 칸 굴리면: Scroll: 0, -1
}
```

함수들: 콜백 방식

- 문자 입력 콜백

- 콜백 함수 등록

- GLFWcharfun **glfwSetCharCallback** (GLFWwindow* window, GLFWcharfun callback);

- callback: 문자 입력 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **CharCallback** (GLFWwindow* window,, unsigned int)); codepoint);

- window: 이벤트 발생한 윈도우

- codepoint: 입력된 문자의 유니코드 포인트

- 사용 예)

```
glfwSetCharCallback(window, CharCallback);
```

```
//--- 콜백 함수 등록
```

```
void CharCallback ( GLFWwindow* window, unsigned int codepoint)
```

```
//--- 등록된 사용자 정의 콜백 함수
```

```
{
```

```
    std::cout << "Unicode = " << codepoint << std::endl;
```

```
        //--- A를 입력 → Unicode = 65
```

```
        //--- 1을 입력 → Unicode = 49
```

```
        //--- a를 입력 → Unicode = 97
```

```
    std::cout << " Unicode = " << (char) codepoint << std:: endl;
```

```
        //--- a를 입력 → Unicode = a
```

```
        //--- 1을 입력 → Unicode = 1
```

```
}
```

- glfwSetKeyCallback()은 '어떤 키를 눌렀는가'를 처리하는 함수
- glfwSetCharCallback()은 '어떤 문자가 입력되었는가'를 처리하는 함수
- 게임 입력은 대부분 Key Callback을 사용하고, 텍스트 입력은 Char Callback을 사용

함수들: 콜백 방식

- 프레임 버퍼 크기 변경될 때 호출: 뷰포트를 새로운 크기에 맞게 다시 설정

- 콜백 함수 등록

- GLFWframebuffersizefun `glfwSetFramebufferSizeCallback` (GLFWwindow* window, GLFWscrollfun callback);

- callback: 프레임 버퍼 크기 변경 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void `FramebufferSizeCallback` (GLFWwindow* window, int width, int height);

- window: 이벤트가 발생한 윈도우

- width/height: 새로운 프레임 버퍼의 너비/높이 (픽셀 단위)

- 사용 예)

```
glfwSetFramebufferSizeCallback ( window, FramebufferSizeCallback);           //--- 콜백 함수 등록
```

```
void FramebufferSizeCallback (GLFWwindow* window, int width, int height)      //--- 등록한 사용자 정의 콜백 함수
{
    glViewport(0, 0, width, height);
}
```

함수들: 콜백 방식

- 윈도우의 크기가 변경될 때

- 콜백 함수 등록

- GLFWwindow* glfwSetWindowSizeCallback (GLFWwindow* window, GLFWwindowfun callback);
 - callback: 문자 입력 콜백 함수

- 실제 호출하는 사용자 정의 콜백 함수

- void **windowSizeCallback** (GLFWwindow* window, int width, int height);
 - window: 이벤트 발생한 윈도우
 - width/height: 실제 윈도우의 폭/높이

- 사용 예) 윈도우의 크기가 변경될 때 화면 비율 계산하기

```
glfwSetWindowSizeCallback (window, window_size_callback);           //--- 콜백 함수 등록
```

```
void window_size_callback (GLFWwindow* window, int width, int height)  //--- 등록한 사용자 정의 콜백 함수
{
    f (height != 0) {
        aspectRatio = (float) (width) / (float)(height);
    }
    std::cout << "Aspect Ratio: " << aspectRatio << std::endl;
}
```

- Window Size는 프로그램에서 사용하는 창의 논리적인 크기
- Framebuffer Size는 실제 렌더링에 사용되는 픽셀 영역의 크기
- 일반적인 환경에서는 둘이 같지만 HiDPI (높은 픽셀 밀도를 가진 디스플레이) 환경에서는 다를 수 있다.
- OpenGL의 glViewport ()는 **일반적으로 Framebuffer Size를 기준으로 설정한다.**

함수들: 콜백 방식

• 윈도우의 화면상 위치가 변경될 때

• 콜백 함수 등록

- GLFWwindowposfun **glfwSetWindowPosCallback** (GLFWwindow* window, GLFWwindowposfun callback);

- Callback: 위치 변경 콜백 함수

• 실제 호출하는 사용자 정의 콜백 함수

- void **windowPosCallback** (GLFWwindow* window, int xpos, int ypos);

- window: 위치가 변경된 윈도우
- xpos/ypos: 윈도우의 새로운 x/y 좌표

• 윈도우가 닫기 요청을 받을 때

• 콜백 함수 등록

- GLFWwindowclosefun **glfwSetWindowCloseCallback** (GLFWwindow* window, GLFWwindowclosefun callback);

• 실제 호출하는 사용자 정의 콜백 함수

- void **windowCloseCallback** (GLFWwindow* window);

함수	역할
glfwWindowShouldClose()	닫기 요청 상태 확인
glfwSetWindowShouldClose()	닫기 요청 상태 설정
glfwSetWindowCloseCallback()	닫기 요청 발생 시 Callback 등록

함수들: 시간

- 시간 관련 함수

- 프로그램 시작 이후의 경과 시간을 초 단위로 반환

- double **glfwGetTime** (void);

- 반환 값: 프로그램 시작 후 경과 시간 (단위는 초). 예를 들어 프로그램을 실행하고 3초후에 호출하면 리턴값은 3.0
 - 애니메이션, 이동, FPS 계산, delta time 계산 등에 사용
 - 사용 예)

- `double time = glfwGetTime();`

- `float x = sin(time);` `//--- 시간이 계속 증가하므로 x도 계속 변화해서 물체가 좌우로 움직이는 애니메이션이 된다.`

- 시간 값 직접 설정

- void **glfwSetTime** (double time);

- Time: 설정 할 시간

- 사용 예)

- `glfwSetTime (0.0);` `//--- 시간을 다시 0초부터 시작 (애니메이션을 재시작할 때 활용 가능)`

함수들: 시간

- 시간 관련 함수

- VSync (Vertical Synchronization, 수직 동기화)

- 게임의 화면 출력 속도 (FPS)를 모니터의 화면 재생 빈도 (Refresh Rate)에 맞추는 기능
 - FPS와 화면재생빈도가 안 맞으면 Tearing (호면 찢김) 현상에 발생할 수 있다.
 - Vsync를 켜면 GPU가 만든 프레임을 모니터의 화면 갱신 타이밍에 맞춰 표시하도록 동기화를 한다.

- VSync 주기 설정

- void **glfwSwapInterval** (int interval);
 - 수직 동기화 (VSync) 설정: 60Hz 모니터 → 60 FPS
 - Interval:
 - 0 – Vsync 사용안하고 가능한 한 빠르게 화면을 갱신
 - 1 – Vsync 사용, 모니터의 한 번의 화면 갱신마다 한번씩 버퍼 교환 (가장 일반적인 설정)

- **OpenGL 컨텍스트를 생성하고 현재 컨텍스트로 만든 다음 호출**

- 사용 예)

```
GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL", nullptr, nullptr);
```

```
glfwMakeContextCurrent(window);
```

```
glfwSwapInterval(1);           //--- 모니터 주기 (예, 60Hz) 에 맞춰 출력한다
```

함수들: 시간

- 사용 예) 애니메이션 만드는 일반적인 방법: 시간이 계속 증가하므로 angle 만큼 계속회전한다

```
while (!glfwWindowShouldClose(window))
{
    double time = glfwGetTime();

    angle = time * 30.0;

    DrawScene (angle);

    glfwSwapBuffers(window);
    glfwPollEvents();
}
```

- 사용 예) 프레임 간 시간 (delta time)을 계산하여 동일한 속도 (speed)로 애니메이션 하는 방법

```
double lastTime = glfwGetTime();

while (!glfwWindowShouldClose(window))
{
    double currentTime = glfwGetTime();

    double deltaTime = currentTime - lastTime;

    lastTime = currentTime;

    position += speed * deltaTime;

    DrawScene();

    glfwSwapBuffers (window);
    glfwPollEvents();
}
```

2. 오픈지엘 기초

- 오픈지엘의 기본 데이터 타입

OpenGL 데이터 형식	표현	C언어 데이터 형식	접미어
GLbyte	8-bit integer	signed char	b
GLshort	16-bit integer	short	s
GLint, GLsizei	32-bit integer	long	i
GLfloat, GLclampf	32-bit floating point	float	f
GLdouble, GLclampd	64-bit floating point	double	d
GLubyte, GLboolean	8-bit unsigned integer	unsigned char	ub
GLushort	16-bit unsigned integer	unsigned short	us
GLuint, GLenum, GLbitfield	32-bit unsigned integer	unsigned long	ui

- 함수 형태

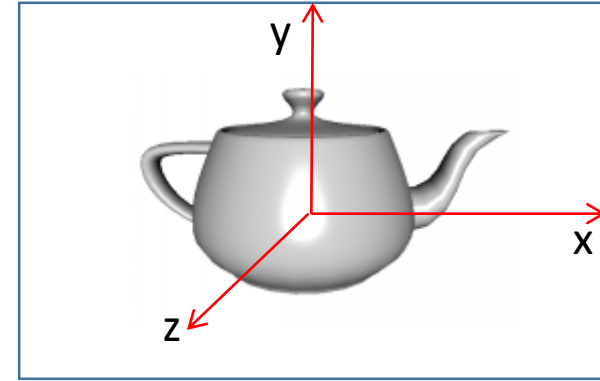
- 예) **glUniform**{1/2/3/4}{b/s/i/f/d/ub/us/ui}{v}: <접두어><기본 명령><접미어>

- gl → 접두어: 함수가 속한 라이브러리
- Uniform → 기본명령: 함수 내용
- 3f → 접미어: 인자 개수와 데이터 형식 (생략 가능)

- 실제 함수는, glUniform1f (vLocation, 1.0f), glUniform2 (vLocation, 0, 0), glUniform3f (vLocation, 0.0f, 0.5f, 1.0f)...

좌표 시스템

- 오픈지엘 좌표계 시스템
 - 화면 중앙이 원점으로 설정
 - 오른손 좌표계: 반 시계 방향이 양의 방향
 - 모든 좌표는 3D 공간의 좌표
 - x축 양의 방향: 왼쪽 → 오른쪽
 - y축 양의 방향: 아래쪽 → 위쪽
 - z축 양의 방향: 화면 안 → 화면 밖
- 초기 OpenGL에서는 화면에 표시되는 기본 좌표 범위를 $[-1.0, 1.0]$ 으로 생각할 수 있다.
 - 즉, NDC (Normalized Device Coordinates) 의 범위이다.
 - 초기 OpenGL에서는 정점을 이 범위 안에 지정하면 화면에 표시할 수 있다.
 - 모든 좌표들은 $[-1.0, 1.0]$ 사이에 놓여야 한다.
 - 투영 공간을 설정하게 되면 좌표계 범위는 달라질 수 있다.



오픈지엘 기본 함수들

- 간단한 OpenGL 함수들:

- 화면 클리어 하기

- void **glClear** (GLbitfield mask)

- 프레임 버퍼의 내용을 초기화

- 컬러 버퍼: GL_COLOR_BUFFER_BIT
 - 깊이 버퍼: GL_DEPTH_BUFFER_BIT
 - 스텐실 버퍼: GL_STENCIL_BUFFER_BIT

- 프레임 버퍼: 화면에 출력하기 위한 데이터를 저장하는 메모리 영역

- 컬러 버퍼: 픽셀의 색상 (RGBA)을 저장 (컬러버퍼는 실제로 front color buffer와 back color buffer가 존재 → 서로 교환되며 화면에 출력하여 화면의 깜빡임을 방지할 수 있다.)
 - 깊이 버퍼: 픽셀의 깊이 (z값)을 저장
 - 스텐실 버퍼: 특정 영역만 렌더링 하도록 마스크 역할

- 화면 바탕색 지정하기

- void **glClearColor** (GLclampf r, GLclampf g, GLclampf b, GLclampf a)

- 윈도우를 칠할 때의 색 지정

- r, g, b: red, green, blue 값
 - a: alpha 값 (1.0값으로 고정)

- 오픈지엘 색상 모델은 RGB 모델, 즉, Red, Green, Blue의 색상을 혼합하여 다른 색상을 만든다.

- 각 R, G, B는 [0.0 , 1.0] 사이의 값으로 설정

오픈지엘 기본 함수들

- 오픈지엘의 장면을 실제 윈도우의 어느 영역에 그릴 것인지 지정하는 함수
 - void **glViewport** (GLint x, GLint y, GLsizei width, GLsizei height);
 - x/y: 뷰포트의 좌측 아래 시작 x/y위치
 - width/height: 뷰포트의 폭/높이

사용 예) 윈도우의 크기가 변경될 때

```
void framebuffer_size_callback (GLFWwindow* window, int width, int height)
{
    glViewport(0, 0, width, height);
}
```

```
glfwSetFramebufferSizeCallback (window, framebuffer_size_callback);
```

- glViewport()는 OpenGL의 NDC 좌표를 윈도우의 실제 렌더링 영역에 매핑하기 위한 viewport의 위치와 크기를 지정하는 함수
- 나중에 뷰포트 변환으로 사용된다.

Legacy OpenGL 함수

- Legacy OpenGL 함수들

- void **glColor3f** (GLfloat red, GLfloat green, GLfloat blue);
 - 색상 지정 함수
 - Red/green/blue: [0.0, 1.0] 범위의 빨간색/초록색/파란색 값
- void **glRectf** (GLfloat x1, GLfloat y1, GLfloat x2, GLfloat y2);
 - 2차원 평면에 직사각형을 그리는 함수
 - x1/y1: 좌측 하단 꼭짓점의 x/y값
 - x2/y2: 우측 상단 꼭짓점의 x/y 값
 - 두 함수 함께 사용 예) 빨간색으로 사각형을 그린다.
`glColor3f(1.0f, 0.0f, 0.0f);`
`glRectf(-0.5f, -0.5f, 0.5f, 0.5f);`

- 위 두 함수는 Modern OpenGL에서는 사용하지 않는 함수
- 이번 강의에서만 사용하도록 하고, 이후에는 셰이더로 정점 속성을 전달해서 도형을 그리도록 한다.
- 위 두 함수를 사용하기 위해서는 컨텍스트를 만들 때 Compatibility Profile을 사용하도록 설정해야 한다. 즉,
`glfwWindowHint (GLFW_CONTEXT_VERSION_MAJOR, 3);`
`glfwWindowHint (GLFW_CONTEXT_VERSION_MINOR, 3);`
`glfwWindowHint (GLFW_OPENGL_PROFILE, GLFW_OPENGL_COMPAT_PROFILE);`
- 이후에는 `glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);`

윈도우 띄우기

```
#include <GL/glew.h>
#include <GL/glfw3.h>
#include <iostream>
```

```
int main() {
    //--- GLFW 초기화
    if (!glfwInit()) {
        std::cerr << "GLFW 초기화 실패!" << std::endl;
        return -1;
    }
```

//--- OpenGL 버전 설정 (예: 3.3 Core Profile)

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

//--- 윈도우 생성

```
GLFWwindow* window = glfwCreateWindow(800, 600, "OpenGL Window", nullptr, nullptr);
if (!window) {
    std::cerr << "윈도우 생성 실패!" << std::endl;
    glfwTerminate();
    return -1;
}
```

//--- 컨텍스트 설정

```
glfwMakeContextCurrent(window);
```

//--- GLEW 초기화

```
glewExperimental = GL_TRUE; // 최신 기능 사용
if (glewInit() != GLEW_OK) {
    std::cerr << "GLEW 초기화 실패!" << std::endl;
    return -1;
}
```

//--- 뷰포트 설정

```
glViewport(0, 0, 800, 600);
```

//--- 메인 루프

```
while (!glfwWindowShouldClose(window)) {
    // 입력 처리
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);
```

// 화면 지우기 (파란색)

```
glClearColor(0.0f, 0.0f, 1.0f, 1.0f); // RGBA (파랑)
glClear(GL_COLOR_BUFFER_BIT);
```

// 버퍼 교체

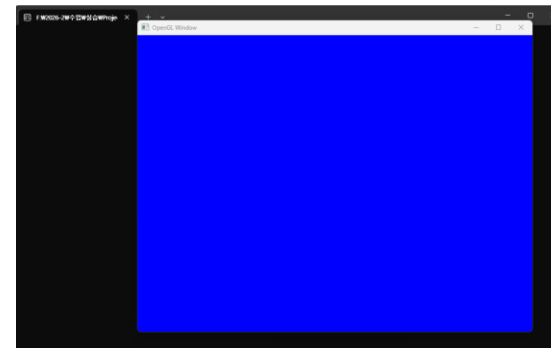
```
glfwSwapBuffers(window);
glfwPollEvents();
```

```
}
```

//--- 종료 처리

```
glfwDestroyWindow(window);
glfwTerminate();
return 0;
```

```
}
```



3. 라이브러리 설치: 1. 다운로드 받기

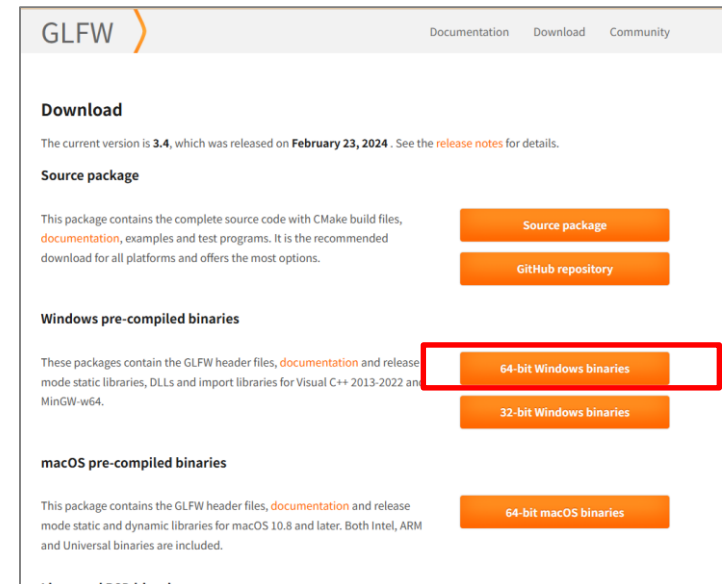
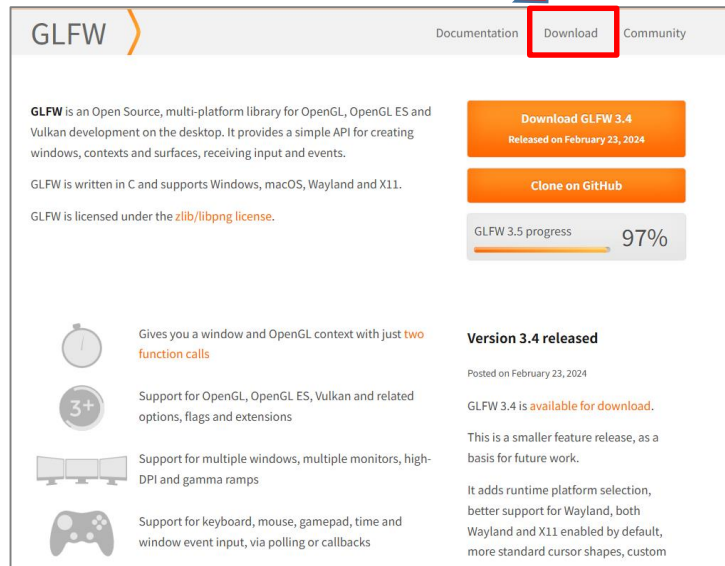
- OpenGL

- OpenGL 표준이 윈도우 시스템에 이미 들어가 있음 (C:\Program Files (x86)\Windows Kits\10 폴더에 저장)

- 라이브러리 다운로드 받기

1) GLFW

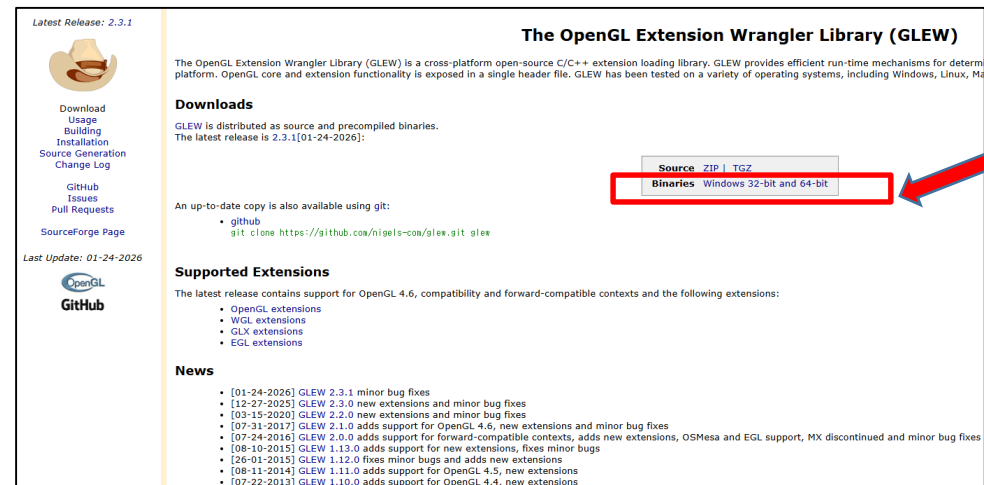
- <https://www.glfw.org/> 에서 Download 로 이동 후, windows pre-compiled binaries 항목에서
- 64-bit Windows binaries 클릭



라이브러리 설치: 1. 다운로드 받기

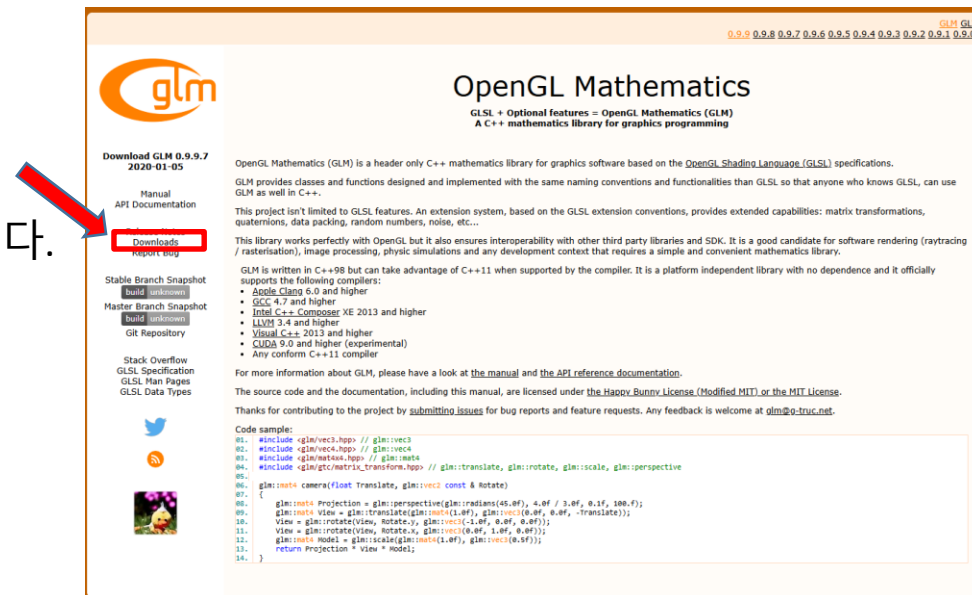
2) GLEW for win32

- <http://glew.sourceforge.net> 에서
- 바이너리 Windows 32비트 및 64비트에서 glew-2.3.1-win32.zip 다운로드 받기
 - 우측 그림과 같이 빨간 네모칸의 링크를 누르면 → OpenGL 확장 Wrangler 라이브러리로 이동한 후 다운로드 된다.



3) GLM

- <https://glm.g-truc.net/0.9.9/index.html> 에서
- glm-1.0.3.zip 다운로드 받기
 - 우측 그림과 같이 빨간 네모칸의 Downloads를 누르면 → 깃허브로 이동한 후 원하는 버전의 라이브러리를 다운로드한다.



라이브러리 설치: 2. 라이브러리 설치하기

1) include 설치 (헤더 파일 설치)

- 설치할 위치: C:\Program Files (x86)\Windows Kits\10\Include\10.0.26100.0\um\gl 폴더
- 설치할 파일들:

① GLFW 헤더 파일들:

- 헤더파일 저장된 위치: `..\glfw-3.4.bin.WIN64\glfw-3.4.bin.WIN64\include\GLFW`
- 저장할 파일들: **폴더 안의 모든 파일들** (`glfw3.h`, `glfw3native.h`)

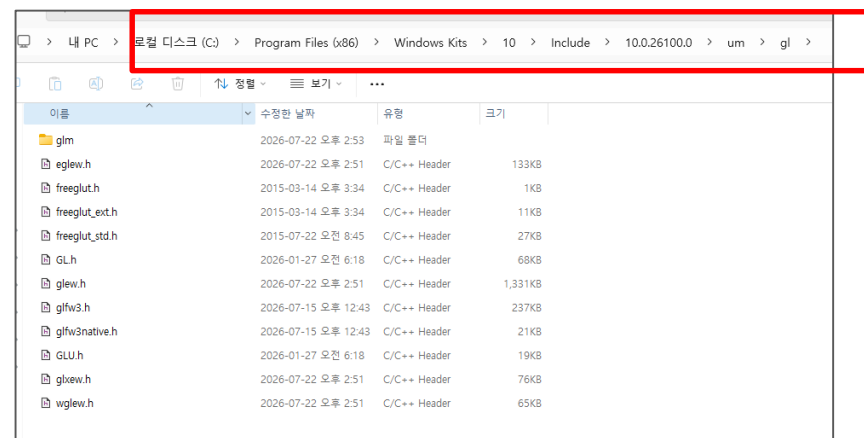
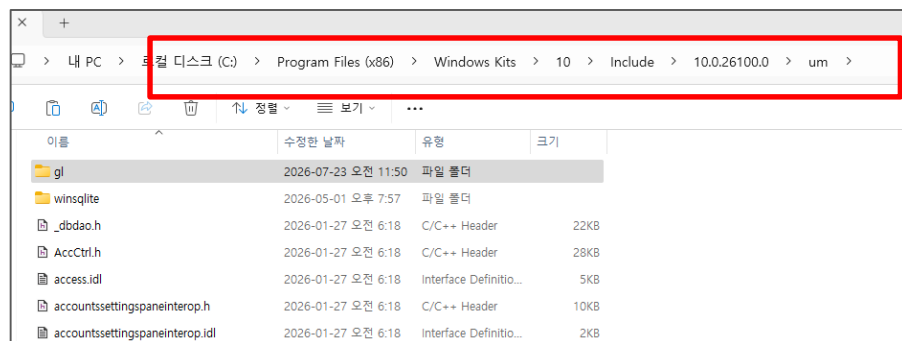
② GLEW 헤더 파일들:

- 헤더파일 저장된 위치: `..\glew-2.3.1-win32\glew-2.3.1\include\GL`
- 저장할 파일들: **폴더 안의 모든 파일들** (`glew.h`, `glxew.h`, `wglew.h`, `eglew.h`)

③ GLM 헤더 파일들:

- 헤더파일 저장된 위치: `..\glm-1.0.3\glm`
- 저장할 파일들: **glm 디렉토리를 통채로**

이 숫자는 개인의 pc에 설치된 윈도우의
버전에 따라 다를 수 있음



라이브러리 설치: 2. 라이브러리 설치하기

2) lib 설치 (정적 라이브러리 설치)

- 설치할 위치: C:\Program Files (x86)\Windows Kits\10\Lib\10.0.22621.0\um\x64 폴더
- 설치할 파일들: *.lib 파일들

① GLFW 라이브러리:

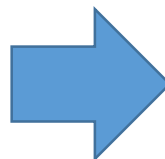
- 라이브러리 저장된 위치: ..\glfw-3.4.bin.WIN64\lib-vc2022 폴더
- 저장할 파일들: glfw3_mt.lib, glfw3.lib, glfw3dll.lib

② GLEW 라이브러리:

- 라이브러리 저장된 위치: ..\glew-2.1.0-win32\glew-2.1.0\lib\Release\x64 폴더
- 저장할 파일들: glew32.lib, glew32s.lib

다운로드 > glfw-3.4.bin.WIN64 > glfw-3.4.bin.WIN64 > lib-vc2022			
이름	수정된 날짜	유형	크기
오늘			
glfw3.dll	2026-07-23 오후 12:06	응용 프로그램 확장	227KB
glfw3.lib	2026-07-23 오후 12:06	Object File Library	690KB
glfw3_mt.lib	2026-07-23 오후 12:06	Object File Library	690KB
glfw3dll.lib	2026-07-23 오후 12:06	Object File Library	31KB

다운로드 > glew-2.3.1-win32 > glew-2.3.1 > lib > Release > x64			
이름	수정된 날짜	유형	크기
오늘			
glew32.lib	2026-07-23 오후 12:06	Object File Library	753KB
glew32s.lib	2026-07-23 오후 12:06	Object File Library	2,836KB



로컬 디스크 (C:) > Program Files (x86) > Windows Kits > 10 > Lib > 10.0.26100.0 > um > x64			
이름	수정된 날짜	유형	크기
esent.lib	2026-01-27 오전 6:18	Object File Library	83KB
evr.lib	2026-01-27 오전 6:18	Object File Library	4KB
exdisp.tlb	2026-01-27 오전 6:18	TLB 파일	49KB
FaultRep.Lib	2026-01-27 오전 6:18	Object File Library	6KB
faxcomex.tlb	2026-01-27 오전 6:18	TLB 파일	93KB
feclient.lib	2026-01-27 오전 6:18	Object File Library	16KB
FhSvcCtl.lib	2026-01-27 오전 6:18	Object File Library	5KB
fileextd.lib	2026-01-27 오전 6:18	Object File Library	359KB
ftLib.lib	2026-01-27 오전 6:18	Object File Library	8KB
fontsub.lib	2026-01-27 오전 6:18	Object File Library	2KB
FrameDyd.Lib	2026-01-27 오전 6:18	Object File Library	204KB
FrameDyn.Lib	2026-01-27 오전 6:18	Object File Library	205KB
freelut.lib	2015-03-14 오후 3:24	Object File Library	36KB
fwpuclnt.lib	2026-01-27 오전 6:18	Object File Library	71KB
fxutility.lib	2026-01-27 오전 6:18	Object File Library	2KB
gameinput.lib	2026-01-27 오전 6:18	Object File Library	2KB
Gdi32.Lib	2026-01-27 오전 6:18	Object File Library	136KB
gdiplus.lib	2026-01-27 오전 6:18	Object File Library	205KB
glew32.lib	2026-07-22 오후 2:51	Object File Library	753KB
glew32s.lib	2026-07-22 오후 2:51	Object File Library	2,836KB
glfw3.lib	2026-07-15 오후 12:43	Object File Library	690KB
glfw3_mt.lib	2026-07-15 오후 12:43	Object File Library	690KB
glfw3dll.lib	2026-07-15 오후 12:43	Object File Library	31KB
gmfs2.lib	2026-01-27 오전 6:18	Object File Library	27KB
GUI32.Lib	2026-01-27 오전 6:18	Object File Library	12KB
GPEdit.lib	2026-01-27 오전 6:18	Object File Library	4KB

라이브러리 설치: 2. 라이브러리 설치하기

3) dll 설치 (동적 라이브러리 설치)

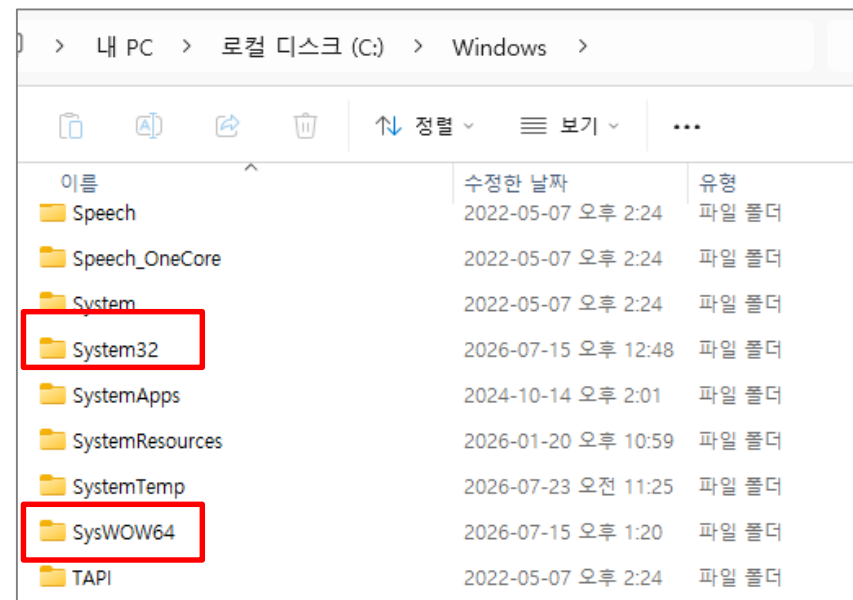
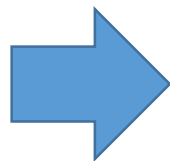
- 설치할 위치: C:\windows\SysWOW64 폴더와 C:\windows\system32 폴더
- 설치할 파일들: *.dll 파일들

① GLFW 라이브러리:

- 라이브러리 저장된 위치: `..\glfw-3.4.bin.WIN64\lib-vc2022`
- 저장할 파일들: `glfw3.dll`

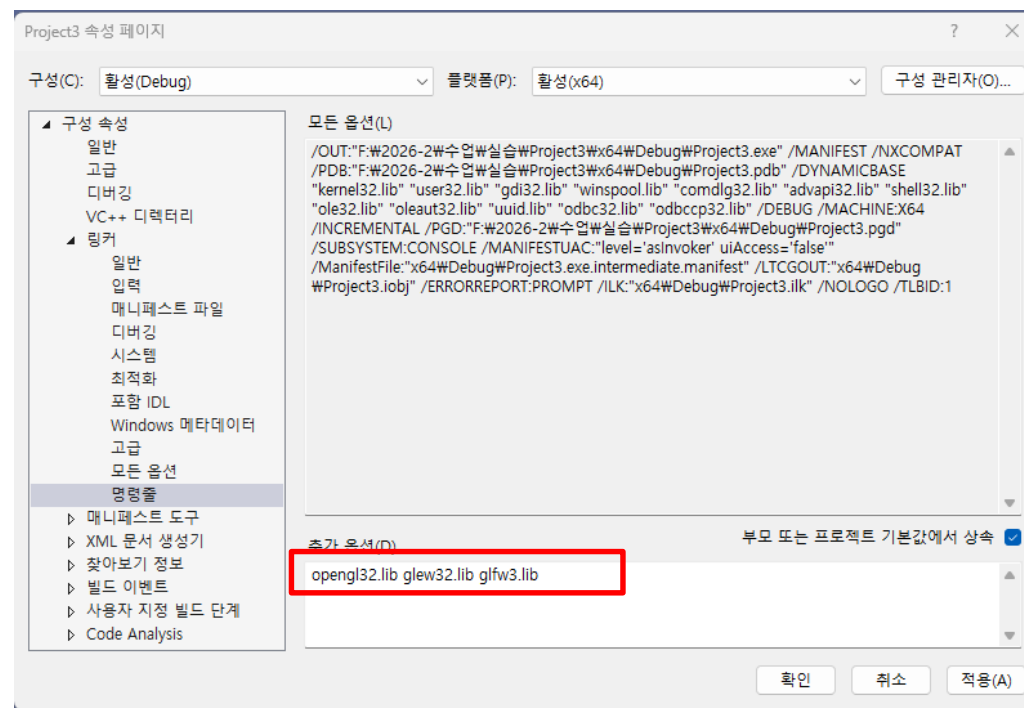
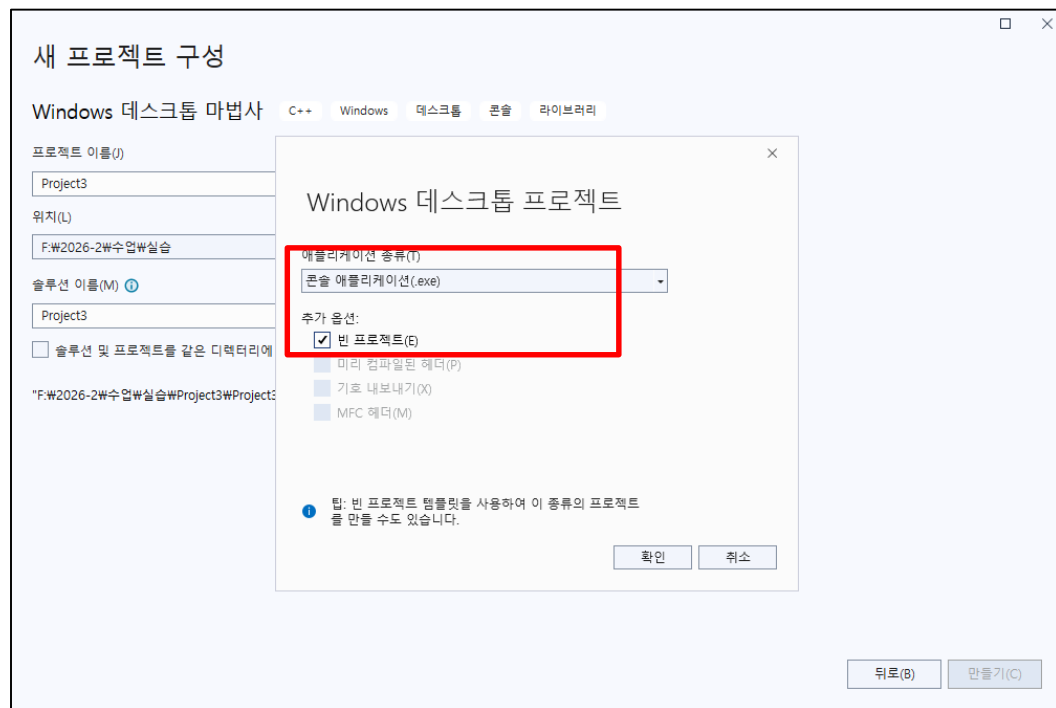
② GLEW 라이브러리:

- 라이브러리 저장된 위치: `..\glew-2.1.0-win32\glew-2.1.0\bin\Release\x64`
- 저장할 파일들: `glew32.dll`



라이브러리 설치 후 프로젝트 만들기

- Visual studio 2022 VC++ 실행
 - 프로젝트 새로 만들기: **콘솔 응용 프로그램**으로 프로젝트 만들기
 - **Windows 데스크톱 마법사** → **콘솔 애플리케이션** → **빈 프로젝트**로 프로젝트 만들기
 - **프로젝트 속성** → **링커** → **명령줄**에 다음과 같이 3개의 라이브러리 추가
 - **opengl32.lib glew32.lib glfw3.lib**



프로젝트 만들기

- Visual studio 2022 VC++ 실행

- 프로그램에 헤더 파일 넣기

```
#include <gl/glew.h>
```

```
#include <gl/glfw3.h>
```

- * 나중에 좌표계 변환 적용할 때, 아래의 헤더파일 추가로 넣기: glm 헤더파일 추가 (폴더 맞춰서 추가하기)

```
#include <gl/glm/glm.hpp>
```

```
#include <gl/glm/ext.hpp>
```

```
#include <gl/glm/gtc/matrix_transform.hpp>
```